

**INSTITUTO
FEDERAL**
Santa Catarina

Revisão de Fundamentos de JavaScript

Programação Frontend II

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

adriano.lima@ifsc.edu.br



cores

#111027

#277756

#16ABCD

#FFF4EC

#C74E23

fontes

Fira Sans Extra Condensed

Ubuntu

Roboto Mono



Tipos de Dados

tipos principais

- Indefinido (undefined)
- Nulo (null)
- Número (number)
- String
- Booleanos
 - falso ("falsy")
 - verdadeiro ("truthy")

exemplos

```
var idade;  
var idade = null;  
var idade = 20;  
var idade = "vinte";  
var idade = true;  
null, undefined, 0, ""  
todo o resto
```



Variáveis

definições

- **var**
 - escopo de função
- **let**
 - escopo de bloco
- **const**
 - escopo de bloco
 - imutável

exemplos

```
var idade = 18;
let nome = "Maria";
const VERSAO = 1.01;

const x = 1;
if (x === 1) {
  let x = 2;
  console.log(x); // Imprime 2
}
console.log(x); // Imprime 1
let x = 3; // ERROR!
```



Template Literals (Strings)

o que são

- **interpola strings com expressões JS**
- pode incluir qualquer número de expressões
- pode usar qualquer um dos caracteres de escape que as strings normais também usam
- pode abranger qualquer número de linhas
- delimitada pelo símbolo da crase (`)
- expressões delimitadas por `${ }`

exemplos

```
let nome = "Mário";  
let saudacao = `Olá, ${ nome }!`;  
console.log(saudação); // Olá, Mário!  
  
const li = document.createElement('li');  
li.textContent = `${aluno.id} - ${aluno.nome}`;
```



Operadores

aritméticos

- + soma
- - subtração
- * multiplicação
- / divisão
- % módulo
- ** exponenciação
- + positivo
- - negativo
- ++ incremento
- -- decremento

exemplos

```
5 + 5 == 10; "5" + 5 == "55";
```

```
6 - 5 == 1;
```

```
5 * 5 == 25;
```

```
5 / 2 == 2.5;
```

```
5 % 2 == 1;
```

```
2 ** 3 == 8;
```

```
+"5" == 5;
```

```
-"-5" == 5;
```

```
var numero = 5; ++numero == 6;
```

```
--numero == 4;
```



Operadores

relacionais

- == é igual a
- != não é igual a
- === é idêntico a (igual e mesmo tipo)
- !== não é idêntico a
- > é maior do que
- >= é maior do que ou igual a
- < é menor do que
- <= é menor do que ou igual a

exemplos

```
alert("5" == 5); // true
```

```
alert(5 != 6); // true
```

```
alert("5" === 5); // false
```

```
alert("5" !== 5); // true
```

```
alert(5 > 4); // true
```

```
alert(5 >= 5); // true
```

```
alert(5 < 6); // true
```

```
alert(5 <= 5); // true
```

Operadores

lógicos

- `!` negação (NOT)
- `&&` conjunção (AND)
- `||` disjunção (OR)

exemplos

```
!0; // true
```

```
x === 0 && y === 0;
```

```
x === 0 || y === 0;
```

x	y	!x	x y	x && y
true	true	false	true	true
true	false	false	true	false
false	true	true	true	false
false	false	true	false	false

Arrays

o que são

- **grupo de valores ordenados que podem ser atribuídos a uma única variável**
- os elementos podem ser de tipos diferentes em um mesmo array
- arrays são dinâmicos
- arrays podem ser esparsos
 - elementos não têm índices contíguos

exemplos

```
let vazio = [];
```

```
let cinco = [5, "cinco", true];
```

```
let esperso = [1, , 3];
```

```
let espersoVazio = [, , ,];
```



Arrays

criando arrays com o construtor

- `new Array()`
 - cria um array vazio (similar ao literal `[]`)
- **sem argumentos**
 - especifica o comprimento (`length`)
- **um argumento numérico**
 - especifica o comprimento (`length`)
- **dois ou mais argumentos numéricos ou pelo menos um não numérico**
 - explicita os elementos do array

exemplos

```
let a = new Array(); // a == []
```

```
let b = new Array(4); // b == [ , , , ]
```

```
let c = new Array(6, "casa", true);  
// c == [6, "casa", true]
```

Arrays

operador de propagação (...)

- insere elementos de um array como elementos de outro array
- funciona com qualquer objeto iterável

exemplos

```
let x = [1, 2, 3];  
let y = [0, ...x, 4]; // y == [0, 1, 2, 3, 4]
```

```
let Algarismos = [..."0123456789"];  
// Algarismos == ["0", "1", "2", "3", "4", "5", "6", "7",  
"8", "9"]
```

```
function removePagto(id) {  
  const pagtosFiltrados = pagtos.filter((pagto) =>  
    pagto.id !== id);  
  
  pagtos.length = 0;  
  pagtos.push(...pagtosFiltrados);  
}
```



Arrays

adicionando e excluindo elementos

- **push()** adiciona valores ao final do array (a[a.length])
- **pop()** remove valores do final do array
- **unshift()** adiciona valores no início do array e move os outros elementos uma posição para trás
- **shift()** remove valores do início do array e move os outros elementos uma posição para frente

exemplos

```
let arrayVazio = [];  
arrayVazio[0] = "zero"; // arrayVazio == ["zero"]  
arrayVazio[1] = "um"; // arrayVazio == ["zero", "um"]
```

```
let numeros = [];  
numeros.push("zero");  
numeros.push("um", "dois");  
numeros.pop(); // numeros == ["zero", "um"]  
numeros.unshift(-1); // numeros == [-1, "zero", "um"]  
numeros.shift(); // numeros == ["zero", "um"]
```



Arrays

métodos

- **forEach()** invoca uma função para cada elemento do array
- **map()** passa cada elemento para a função invocada e retorna um array com os resultados da função
- **filter()** retorna um subconjunto do array baseado na função filtro
- **find(); findIndex()** similar a **filter()**, mas param no primeiro true

exemplos

```
let dados = [1,2,3,4,5], soma = 0;

dados.forEach((v, i, a) => { a[i] = v + 1; });

let incremento = dados.map( v => v + 1; );

const pares = dados.filter( v => v % 2 === 0 );

var primeiroPar = dados.find( x => x % 2 === 0 );

let indice = dados.findIndex( x => x % 5 === 0 );
```



Arrays

métodos

- **concat()** concatena arrays
- **slice()** retorna um subconjunto do array
- **splice()** insere e remove elementos no próprio array
- **sort()** ordena os elementos no próprio array

exemplos

```
let a = [1,2,3];  
a.concat(4, 5); // [1,2,3,4,5]  
a.concat([6,7],[8,9]); // [1,2,3,6,7,8,9]  
a.concat([10,[11,12]]); // [1,2,3,10,[11,12]]  
a = a.concat(4, 5);  
  
a.slice(0, 3); // [1,2,3]  
a.slice(3); // [4,5]  
a.slice(1, -1); // [2,3,4]  
a.slice(-3, -2); // [3]
```

Arrays

métodos

- **concat()** concatena arrays
- **slice()** retorna um subconjunto do array
- **splice()** insere e remove elementos no próprio array
- **sort()** ordena os elementos no próprio array

exemplos

```
let m = [1,2,3,4,5,6,7,8];  
m.splice(4); // m == [1,2,3,4]; => [5,6,7,8]  
m.splice(1,2); // m == [1,4]; => [2,3]
```

```
let n = [1,2,3,4,5];  
n.splice(2,0,"a","b") // n == [1,2,"a","b",3,4,5]; => []  
n.splice(2,2,[1,2],3); // n == [1,2,[1,2],3,3,4,5]; =>  
["a","b"]
```

```
let frutas = ["banana", "maçã", "açaí"];  
frutas.sort(); // frutas == ["açaí", "banana", "maçã"]  
[3, 2, 1].sort(function(a, b){return a - b;});
```



Condicionais

o que são

- **executam instruções condicionalmente**
- **if** permite a execução ou não de um bloco de comandos
- **if-else** permite a seleção entre dois blocos de comandos distintos para execução
- **switch** permite a seleção entre dois ou mais blocos de comandos distintos para execução

exemplos

```
if (nomeUsuario == null) {  
    nomeUsuario = "Zé Ninguém";  
}  
  
if (n === 1) {  
    alert("Você tem 1 mensagem!");  
} else {  
    alert("Você tem ${n} mensagens!");  
}  
  
switch (mes) {  
    case 1:  
        alert("janeiro");  
        break;  
    case 2:  
        alert("fevereiro");  
        break;  
    default: // opcional  
        alert("Mês inválido");  
        break;  
}
```



Laços

o que são

- **executam instruções repetidamente**
- **while** permite a repetição de um bloco de instruções enquanto a expressão for verdadeira
- **do-while** permite a repetição de um bloco de instruções pelo menos uma vez
- **for** permite a repetição finita da execução de um bloco de instruções
- **for-of** permite a repetição de um bloco de instruções para cada elemento de uma coleção

exemplos

```
let contador = 0, indice = 0;
while (count < 10) {
  console.log(contador);
  contador++;
}

if (arr.length > 0) {
  do {
    console.log(arr[indice]);
  } while (++indice < arr.length);
}
```

Laços

o que são

- **executam instruções repetidamente**
- **while** permite a repetição de um bloco de instruções enquanto a expressão for verdadeira
- **do-while** permite a repetição de um bloco de instruções pelo menos uma vez
- **for** permite a repetição finita da execução de um bloco de instruções
- **for-of** permite a repetição de um bloco de instruções para cada elemento de uma coleção

exemplos

```
for (let i = 0; i < 10; i++) {  
  console.log(i);  
}
```

```
let dados = [1, 2, 3, 4, 5, 6, 7, 8, 9];  
let soma = 0;  
for (let elemento of dados) {  
  soma += elemento;  
}
```

```
let palavra = "programa";  
for (let letra of palavra) {  
  console.log(letra);  
}
```



Funções

o que são

- **blocos de código definidos uma vez, mas podem ser executados inúmeras vezes**

- declaração de função
- expressão de função
- função seta
- função aninhada

exemplos

```
function somaNumeros(a, b) {  
  return a + b;  
}
```

```
const quadrado = function (x) {  
  return x * x;  
};
```

```
const soma = (x, y) => {  
  return x + y;  
};
```

```
botao.addEventListener("submit", (evento) => {  
  evento.preventDefault();  
  const membro = { id, nome };  
  membros.push(membro);  
});
```



Objetos

o que são

- **coleções de propriedades, cada uma com um nome e um valor**
- propriedades podem ser adicionadas e excluídas dinamicamente
- nomes geralmente são strings
- valores podem ser primitivos ou objetos

exemplos

```
let vazio = {};
```

```
let livro = {  
  titulo: "JavaScript",  
  subtítulo: "O Guia Definitivo",  
  para: "todas as audiências",  
  autor: {  
    "primeiro nome": "David",  
    sobrenome: "Flanagan",  
  },  
};
```



Objetos

propriedades

- acessadas/escritas pelos operadores ponto (.) e chaves ([])
- propriedades inexistentes são criadas automaticamente quando escritas
- podem ser acessadas condicionalmente (?. e ?. [])
- pode ser excluídas (delete)

exemplos

```
let autor = livro.autor;  
  
autor.sobrenome = "de Assis";  
  
autor["primeiro nome"] = "Machado";  
  
livro["titulo"] = "Dom Casmurro";  
  
livro.edicao = 7;  
  
let local = livro.autor.nascimento?.local; // undefined  
  
let nome = livro.autor?.["nome"]; // undefined  
  
delete livro.edicao;
```



Objetos

serialização

- conversão do estado de um objeto em uma string da qual ele pode ser restaurado posteriormente
- **JSON** "JavaScript Object Notation"
 - **stringify()** serializa o objeto
 - **parse()** restaura o objeto

exemplos

```
let o = {x: 1, y: {z: [false, null, ""]}};  
  
let s = JSON.stringify(o);  
// s == '{"x":1,"y":{"z":[false,null,""]}}'  
  
let p = JSON.parse(s);  
// p == {x: 1, y: {z: [false, null, ""]}}
```



Desestruturação

o que é

- atribuição via desestruturação
- possibilita extrair dados de arrays ou objetos em variáveis distintas
- pode ser feita com arrays ou com objetos
- os nomes das variáveis desestruturadas não precisam ser os mesmos dos atributos dos objetos

exemplos

```
const a, b, rest;  
[a, b] = [1, 2];  
console.log(a); // 1  
console.log(b); // 2  
  
[a, b, ...rest] = [1, 2, 3, 4, 5];  
console.log(rest); // [3, 4, 5]  
  
const x = [1, 2, 3, 4, 5];  
const [y, z] = x;  
console.log(y); // 1  
console.log(z); // 2  
  
const o = { p: 42, q: true };  
const { r, s } = o;  
console.log(r); // 42  
console.log(s); // true
```



Manipulando o DOM

acessando elementos

■ métodos JS utilizam seletores CSS

- `querySelector()`
- `querySelectorAll()`
- `getElementById()`
- `getElementsByName()`
- `getElementsByTagName()`
- `getElementsByClassName()`

exemplos

```
document.getElementById("nav");  
// document.querySelector("#nav");  
  
document.getElementsByName("cor");  
// document.querySelectorAll("[name='cor']");  
  
document.getElementsByTagName("h2");  
// document.querySelectorAll("h2");  
  
document.getElementsByClassName("spin");  
// document.querySelectorAll(".spin");
```



Manipulando o DOM

acessando atributos

■ utilizando métodos

- `getAttribute()`
- `setAttribute()`
- `hasAttribute()`
- `removeAttribute()`

■ acessando propriedades

exemplos

```
const link = document.querySelector('a');

const endereco = link.getAttribute('href');
// endereco = "http://bing.com"

link.setAttribute('href', 'http://google.com');
// <a href="http://google.com">

const nome = document.querySelector('input');

nome.hasAttribute('required');
// true/false

const botao = document.querySelector('.botao');

nome.removeAttribute('disabled');
// ativa o botão
```



Manipulando o DOM

acessando atributos

■ utilizando métodos

- `getAttribute()`
- `setAttribute()`
- `hasAttribute()`
- `removeAttribute()`

■ acessando propriedades

exemplos

```
const link = document.querySelector('a');

const endereco = link.getAttribute('href');
// endereco = "http://bing.com"

link.href = 'http://google.com';
// <a href="http://google.com">

const nome = document.querySelector('input');

nome.hasAttribute('required');
// true/false

const botao = document.querySelector('.botao');

nome.removeAttribute('disabled');
// ativa o botão
```



Manipulando o DOM

acessando classes

■ caso especial de atributo

■ classList

- add()
- remove()
- toggle()
- contains()

exemplos

```
const botao = document.querySelector('.botao');
```

```
botao.classList.add('ativo');  
// adiciona classe
```

```
botao.classList.remove('ativo');  
// remove classe
```

```
botao.classList.toggle('ativo');  
// adiciona se não tiver, remove se tiver
```

```
botao.classList.contains('ativo');  
// true/false
```



Manipulando o DOM

acessando classes

■ HTML

- innerHTML
- outerHTML

■ texto

- textContent
- innerText

exemplos

```
<p>Parágrafo com um <a href="foo.html">link</a> aqui.</p>
```

```
const paragrafo = document.querySelector('p');
```

```
paragrafo.innerHTML;  
// Parágrafo com um  
// <a href="foo.html">link</a> aqui.
```

```
paragrafo.outerHTML;  
// <p>Parágrafo com um  
// <a href="foo.html">link</a> aqui.</p>
```

```
paragrafo.textContent;  
// Parágrafo com um link aqui.
```



Manipulando o DOM

criando, inserindo e removendo elementos

■ criando elementos

- createElement()

■ inserindo elementos

- append()
- prepend()
- after()
- before()

■ removendo elementos

- replaceWith()
- remove()

exemplos

```
const item = document.createElement('li');
item.textContent('Bife acebolado');
// cria um item de lista
```

```
const cardapio = document.querySelector('ul');
cardapio.append(item);
// insere no final
cardapio.prepend(item);
// insere no início
cardapio.children[1].after(item);
// insere depois do filho da posição 1
const novoItem = item.cloneNode(true);
cardapio.children[2].replaceWith(novoItem);
// substitui o filho da posição 2
cardapio.children[2].remove();
// remove o filho da posição 2
```



Manipulando o DOM

acessando estilos

- adicionando/removendo classes
- alterando a propriedade style
- obtendo os estilos calculados pelo navegador
- acessando as folhas de estilo diretamente

exemplos

```
// CSS
.escondida { display: none; }

const imagem = document.querySelector('img');
imagem.classList.toggle('escondida');

function posiciona(imagem, x, y) {
  imagem.style.display = "block";
  imagem.style.position = "absolute";
  imagem.style.left = `${x}px`;
  imagem.style.top = `${y}px`;
}
```



Manipulando o DOM

acessando estilos

- adicionando/removendo classes
- alterando a propriedade style
- obtendo os estilos calculados pelo navegador
- acessando as folhas de estilo diretamente

exemplos

```
const main = document.querySelector("main");
const item = document.querySelector("#item");

const estiloMain = window.getComputedStyle(main);

if (parseFloat(estiloMain.width) < 500) {
  item.style.flexDirection = "column";
} else {
  item.style.flexDirection = "row";
}
```



Manipulando o DOM

acessando estilos

- adicionando/removendo classes
- alterando a propriedade style
- obtendo os estilos calculados pelo navegador
- acessando as folhas de estilo diretamente

exemplos

```
function setTheme(name) {  
  const link = document.createElement("link");  
  link.id = "theme";  
  link.rel = "stylesheet";  
  link.href = `temas/${name}.css`;  
  
  const tema = document.querySelector("#theme");  
  
  if (tema) {  
    tema.replaceWith(link);  
  } else {  
    document.head.append(link);  
  }  
}
```



Manipulando o DOM

acessando estilos

- adicionando/removendo classes
- alterando a propriedade style
- obtendo os estilos calculados pelo navegador
- acessando as folhas de estilo diretamente

exemplos

```
document.head.insertAdjacentHTML(  
  "beforeend",  
  "<style>body{transform:rotate(180deg)}</style>",  
);
```



Manipulando o DOM

tratando eventos

- **registros do que acontece no documento ou no navegador**
 - **tipo** especifica que tipo de evento ocorreu
 - **alvo** objeto/elemento onde o evento ocorreu ou com o qual está associado
 - **manipulador/escuta** função que manipula ou responde a um evento
 - **objeto** associado ao evento e contém detalhes sobre ele
 - **propagação** processo pelo qual o navegador decide quais objetos acionam os manipuladores de eventos

exemplos

```
const form = document.querySelector("form");

form.addEventListener("submit", (evento) => {
  evento.preventDefault();
  cadastraPagto();
});
```



Manipulando o DOM

registrando manipuladores

- como atributos
- como propriedades
- com o métodos
 - `addEventListener()`
 - `removeEventListener()`

exemplos

```
<!-- HTML -->
<button onclick="passarImg(1)">Próxima</button>
<input onFocus="iluminaCampo()">

// JS
form.onsubmit = cadastraPagto();
link.onmouseover = e => e.preventDefault();

form.addEventListener("submit", evento => {
  evento.preventDefault();
  cadastraPagto();
});

form.removeEventListener("submit", cadastraPagto());
```



Persistência

localStorage e sessionStorage

- objetos Storage
- propriedades devem ser strings
- propriedades persistem
- escopo na mesma origem
- métodos
 - clear()
 - getItem()
 - setItem()
 - deleteItem()

exemplos

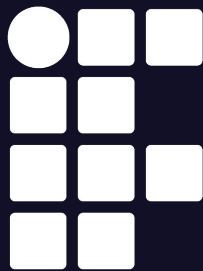
```
let nome = localStorage.nomeusuario;

if (!nome) {
  nome = prompt("Qual é o seu nome?");
  localStorage.nomeusuario = nome;
}

localStorage.clear();

localStorage.deleteItem(nome);
delete localStorage.nome;
```





**INSTITUTO
FEDERAL**
Santa Catarina

Revisão de Fundamentos de JavaScript

Programação Frontend II

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

adriano.lima@ifsc.edu.br

